Stutter: Evidence-Based Validation of Linux Game-Performance Tuning

Linux gaming performance tuning often relies on community advice rather than controlled evidence. Players commonly change CPU governors, scheduler settings, Proton/Wine options, thread affinity, launch flags, or kernel features in an attempt to improve smoothness. However, noisy real-world games make it difficult to know whether a change genuinely improves performance or only appears to help due to normal run-to-run variation. This can lead to wasted effort, misplaced confidence, and even regressions when well-intentioned tweaks are applied without controlled validation.

This problem affects not only Linux gamers, but also game developers and tool authors, who need reliable ways to evaluate performance changes under real-world conditions. Average FPS alone is not enough: frame pacing, stutter, 1% lows, p99 frametime, and scheduler-visible delay often better represent player-visible smoothness, especially in Proton/Wine game stacks involving game threads, Wine helpers, DXVK/VKD3D workers, Gamescope, Steam runtime processes, and compositor-related work.

This project investigates that problem by developing `stutter`, an evidence-based profiling and comparison workflow for Linux games. Existing tools such as MangoHud, `perf`, GPU telemetry utilities, and kernel tracing tools expose useful parts of the problem, but they are usually separate and do not by themselves provide a complete workflow for validating tuning recommendations. `stutter` is intended to combine several evidence streams into one repeatable validation workflow rather than replace those tools. The project combines scheduler-aware eBPF profiling, frame-time data, process-tree classification, data-quality checks, profile explainability, and repeated A/B comparison to decide whether a tuning change is supported by evidence.

Research question

Can scheduler-aware profiling and repeated A/B measurement provide enough evidence to validate, reject, or mark as inconclusive Linux game-performance tuning hypotheses in noisy real-world workloads?

Current prototype

A working prototype, `stutter`, has already been developed in Rust using Aya/eBPF. It currently includes:

- eBPF-based runnable-latency profiling, measuring scheduler wakeup-to-dispatch delay per thread.
- Process-tree tracking for complex Linux game stacks, including Proton/Wine, launchers, compositors, and helper processes.
- Thread/task classification and profile explainability, showing which tasks a tuning profile would affect and why the tool matched them.
- MangoHud frame-timing ingestion and correlation with scheduler evidence.
- GPU, IRQ, block I/O, CPU-frequency, and data-quality artifact support.
- A guarded tuning workflow for safe observe, suggest, explain, apply/test, compare, revert, or decline behavior.
- A report/artifact pipeline for preserving benchmark evidence.

The main difference from using tools such as MangoHud or `perf` individually is that `stutter` attempts to connect measurement, hypothesis generation, profile auditing, repeated comparison, and recommendation handling into one auditable workflow.

In the final methodology, profile explanation would be run before A/B tuning so that the affected threads, matching rules, and proposed CPU masks are auditable before any measurement comparison is made.

Preliminary case study

A preliminary real-world case study has already been completed using *Kingdom Come: Deliverance 1* under GE-Proton on Linux/Wayland/Gamescope. The prototype collected five formal baseline runs and a five-iteration paired A/B comparison of a CPU-affinity tuning hypothesis. The archived A/B run was paired but not counterbalanced: the online baseline was measured before the tuned profile in every iteration, so the case study is treated as preliminary low-confidence evidence and as motivation for enforcing alternating order in the FYP methodology.

The tested profile was plausible: it placed game/Wine tasks on CPUs `1-5,7-11` and Gamescope/runtime tasks on CPUs `0,6`. However, the profile was not validated. The online baseline had a lower primary diagnostic score in all five paired A/B iterations, with the order caveat above. This is a useful result because it demonstrates that the workflow can decline an unsupported recommendation rather than turning a plausible tweak into advice.

The case study also exposed the need for better profile auditability. In response, a profile-explainability feature was added so that the tool can show which profile rules matched important threads such as `RenderThread`, `ClothingRaycast`, `dxvk-submit`, and `wineserver` before a profile is applied.

This case study is not presented as a universal claim about KCD1 performance or CPU affinity. Its value is methodological: it shows that the prototype can:

- collect real workload evidence from a complex Proton/Wine workload;
- test a scoped tuning hypothesis with repeated A/B comparison and explicit order/counterbalancing checks;
- quantify uncertainty in a noisy game workload;
- decline an unsupported tuning recommendation instead of producing a false-positive tuning claim.

The case study also showed that small effects in this type of workload may require substantially more data to measure precisely: depending on the metric, the tool estimated roughly 18–30+ runs per condition for small-effect detection.

Existing work and assessment boundary

The current `stutter` prototype and KCD1 case study are preliminary work used to demonstrate feasibility before supervision. The proposed FYP would not be assessed as simply “the existing tool” or as a general-purpose Linux optimizer. The assessed contribution would be the formalization and evaluation of a bounded methodology: a reproducible benchmark protocol, profile explainability as a required pre-tuning step, repeated A/B comparison, uncertainty-aware reporting, and conservative recommendation verdicts.

The core evaluated tuning area would be CPU affinity and process/thread placement. Existing support for broader areas such as daemon control, GPU power, IRQ affinity, VM knobs, remote/agent paths, system-wide autotuning, scheduler replacement, and wide hardware/game coverage would be treated as

background implementation, optional extension, or future work unless explicitly agreed with the supervisor.

Proposed FYP scope

The proposed Final Year Project would build on this prototype and focus on turning it into a rigorous game-performance evaluation methodology. The main work would be to:

- refine the benchmark methodology for repeatable Linux game-performance testing;
- evaluate scoped tuning hypotheses using repeated baseline-versus-tuned runs;
- use robust statistics, confidence intervals, effect sizes, and bootstrap or non-parametric methods suitable for noisy frame-time distributions;
- improve automated reporting and explainability so that recommendations are auditable;
- clearly separate validated improvements, regressions, inconclusive results, and unsafe or unsupported changes.

To keep the project achievable, the evaluation would focus on a limited set of reversible tuning areas. CPU affinity and process/thread placement would be the primary tuning area, with selected process-local scheduler controls such as `uclamp` considered only if appropriate. Broader or higher-risk system changes, such as persistent IRQ affinity changes or replacing the system scheduler, would be treated as optional extensions or future work.

The current prototype and KCD1 case study show that the idea is viable. The FYP would formalize, narrow, evaluate, and present the methodology as a complete project rather than trying to become a universal Linux game optimizer.

Success criteria

The project would be considered successful if:

- the tool can record reproducible scheduler/frame evidence for at least one real Linux game workload;
- the workflow can compare baseline and tuned configurations using repeated runs and uncertainty-aware statistics;
- the report can distinguish validated improvements, regressions, and inconclusive or unsupported results;
- the methodology is documented clearly enough for another technical user to reproduce the experiment;
- the evaluation reports internal diagnostic scores alongside raw frame-time and scheduler metrics, such as p95/p99/max frametime, outlier counts, scheduler threshold counts, effect sizes, and confidence intervals;
- the system avoids recommending a tuning change when the collected evidence does not support it.

Expected deliverables

- A working profiling and comparison prototype for Linux game workloads.
- A controlled benchmark methodology for repeatable game-performance testing.
- A statistical A/B comparison pipeline for heavy-tailed frame-time data.

- A primary real-world benchmark dataset using repeated runs of a reproducible game route, with optional additional workloads if scope allows.
- Documentation for reproducing experiments, including setup notes, command examples, and example configurations.
- A report showing whether selected tuning recommendations produce measurable improvements, regressions, or inconclusive results.
- A discussion of how evidence-based tuning could help Linux gamers, game developers, and performance-tool authors reason about frame pacing and stutter.

Supervision questions

1. Is this appropriately scoped for a Computer Games Development Final Year Project, or should it be narrowed further?
2. Should the final project emphasize tool implementation, empirical benchmarking, statistical validation, or a balance of these?
3. Does this topic fit your supervision interests, particularly around game performance, systems, benchmarking, or real-time behavior?